

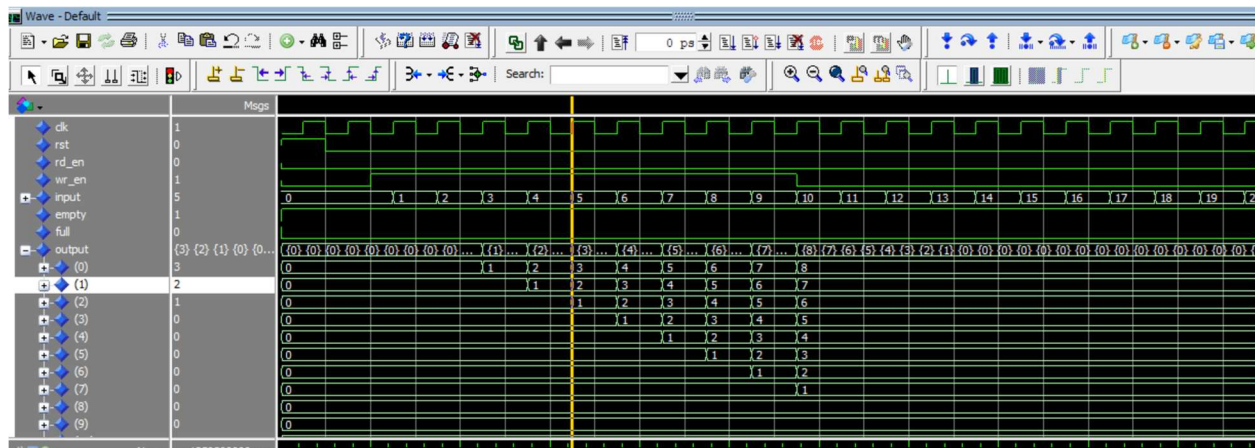
Reconfigurable Computing Final Project Report

Group members: Luke Saleh and Jonas Wood

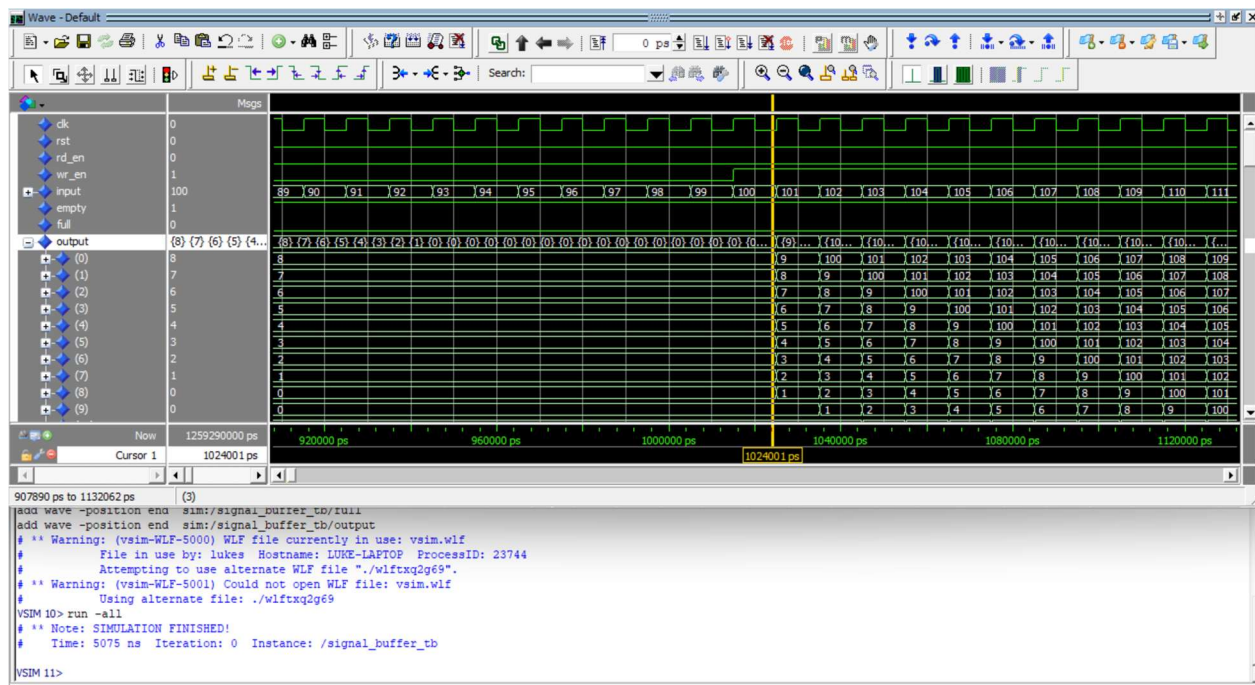
Convolution Portion:

Signal Buffer:

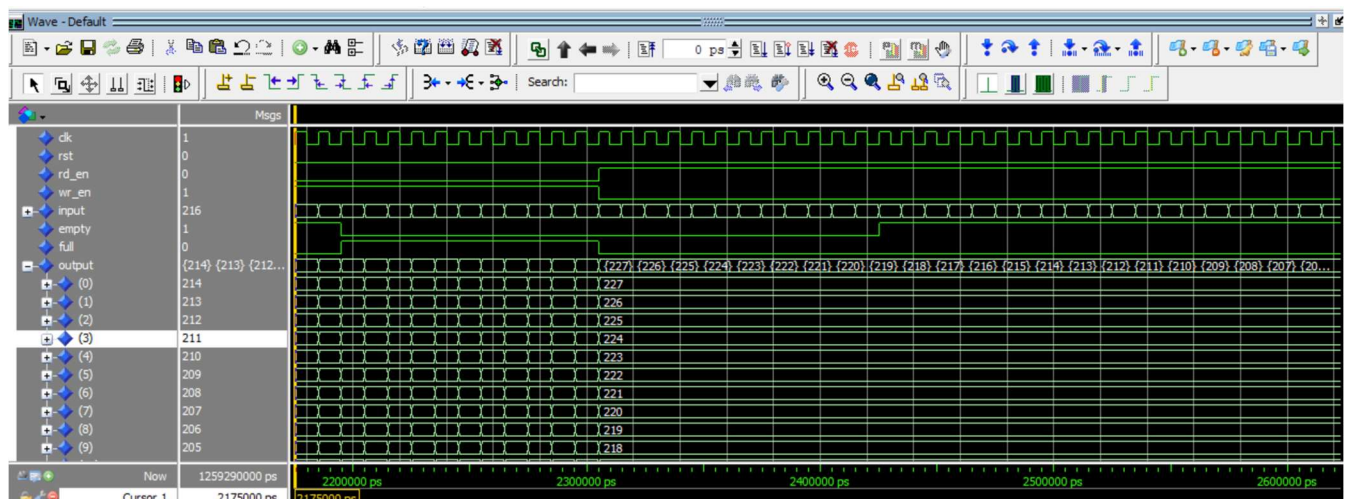
The signal buffer implements a sliding window buffer using the window data type given in the user_pkg file. On each clock cycle, if wr_en is asserted, new input data is loaded into the first element of the register chain and existing data is shifted forward, thereby forming a continuous pipeline of samples. The count signal keeps track of how many valid data points are currently stored, while empty and full signals indicate whether the buffer is underfilled or full, respectively, based on the number of elements relative to the pre-defined kernel size. By chaining registers and controlling read and write enables, the design provides a sliding window of output samples ready for subsequent processing. The following images show a custom testbench run of the signal buffer, where the buffer was run for a little over 500 clock cycles and had the read and write enables set to high and low at various points.



The figure above shows a successful write into the signal buffer. When the write enable goes high, data goes into the buffer and the data is continuously shifted through the buffer while the new data arrives. It then stops being written when the write enable is set to low.



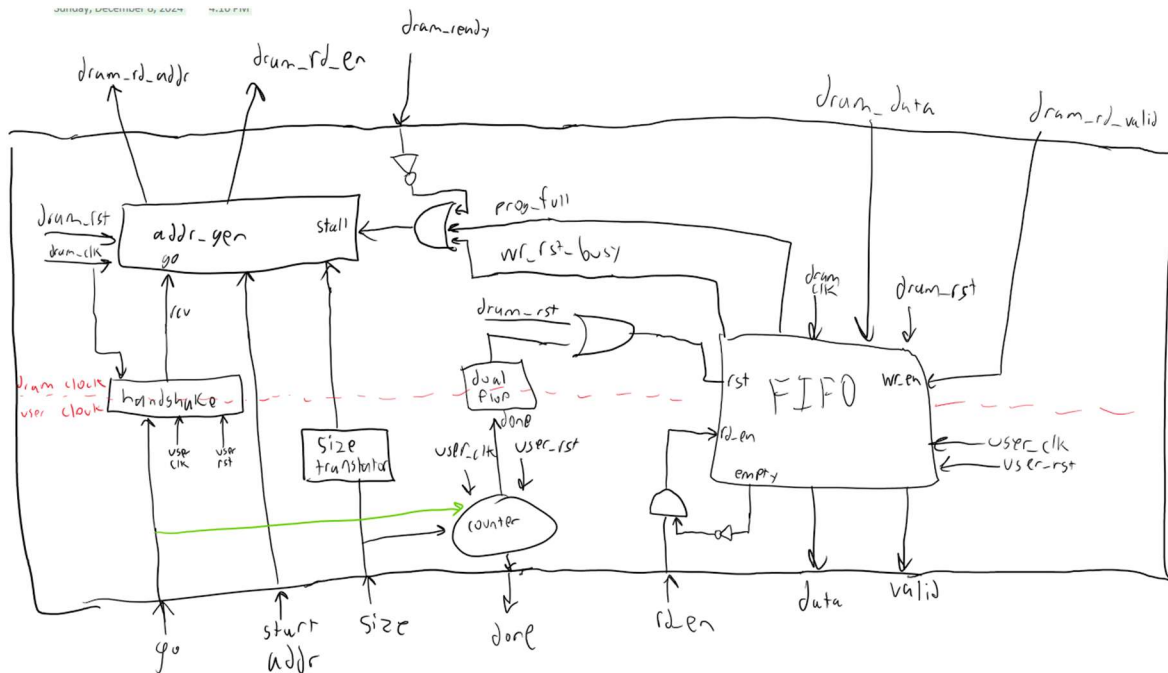
This second figure above shows a second write into the buffer. The previous write enable left the input of 9 waiting to enter the buffer, so this is the first value in the buffer. From there, the buffer receives continuous writes with the new data.



This third image shows the signal buffer first reaching a full state after enough data is written to get to a completely full buffer. This leads to the full flag being asserted, since the count has now reached the full size of the buffer. Afterwards, the rd enable is asserted which causes the full flag to be set to low since the buffer can now be read from and is no longer technically “full”.

DRAM_RD Portion:

Block Diagram:



The figure above shows the block diagram of the entire `dram_rd` component. There are a few primary blocks, along with some additional logic performed on intermediary signals. The main components are the handshake, address generator, size translator, dual flop, counter, and the FIFO. To test the entity, the following process occurs:

1. Go is set to high, triggering a test
2. Go passes through the handshake to cross clock domains into the `dram_clk` space
3. Size is translated according to specifications in the lecture video, with cases for odd or even sizes
4. Start address and translated size are held in a register until the handshake allows go to pass through
5. FIFO takes in data until a stall is set on the address generator

Overall Results:

The testbench passed in both simulation and on the FPGA servers. This can be seen in the images below:

```
App.h Board.cpp Board.o dram_test.bit DramTest.h Lab2 Lab4 main.cpp Makefile Timer.h zed_app
[luksaleh@ece-b312-recon2 ~]$ zed_schedule.py ./zed_app dram_test.bit
Searching for available board...
Starting job "./zed_app dram_test.bit" on board 192.168.1.128:
Programming FPGA...SUCCESS
Testing transfers to/from address 0...SUCCESS
Testing max transfer size...SUCCESS
Testing random sizes and addresses...SUCCESS
[luksaleh@ece-b312-recon2 ~]$ zed_schedule.py ./zed_app dram_test.bit
Searching for available board...
Starting job "./zed_app dram_test.bit" on board 192.168.1.128:
Programming FPGA...SUCCESS
Testing transfers to/from address 0...SUCCESS
Testing max transfer size...SUCCESS
Testing random sizes and addresses...SUCCESS
[luksaleh@ece-b312-recon2 ~]$ zed_schedule.py ./zed_app dram_test.bit
Searching for available board...
Starting job "./zed_app dram_test.bit" on board 192.168.1.105:
Programming FPGA...SUCCESS
Testing transfers to/from address 0...SUCCESS
Testing max transfer size...SUCCESS
Testing random sizes and addresses...SUCCESS
[luksaleh@ece-b312-recon2 ~]$ zed_schedule.py ./zed_app dram_test.bit
Searching for available board...
Starting job "./zed_app dram_test.bit" on board 192.168.1.105:
Programming FPGA...SUCCESS
Testing transfers to/from address 0...SUCCESS
Testing max transfer size...SUCCESS
Testing random sizes and addresses...SUCCESS
[luksaleh@ece-b312-recon2 ~]$

) launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:12 . Memory (MB): peak = 1674.949 ; gain = 9.023
) run_all
Note: Test0 completed. Total Errors: 0
Time: 1265 ns Iteration: 0 Process: /wrapper_tb/line__242 File: C:/Users/luks/Desktop/Schoolwork/EEL5721/git/Reconfig/Projec
Note: Test1 completed. Total Errors: 0
Time: 2465 ns Iteration: 0 Process: /wrapper_tb/line__242 File: C:/Users/luks/Desktop/Schoolwork/EEL5721/git/Reconfig/Projec
Note: Test2 completed. Total Errors: 0
Time: 3565 ns Iteration: 0 Process: /wrapper_tb/line__242 File: C:/Users/luks/Desktop/Schoolwork/EEL5721/git/Reconfig/Projec
Note: Test3 completed. Total Errors: 0
Time: 294475 ns Iteration: 0 Process: /wrapper_tb/line__242 File: C:/Users/luks/Desktop/Schoolwork/EEL5721/git/Reconfig/Projec
Note: TestBig completed. Total Errors: 0
Time: 795535 ns Iteration: 0 Process: /wrapper_tb/line__242 File: C:/Users/luks/Desktop/Schoolwork/EEL5721/git/Reconfig/Projec
Note: SIMULATION FINISHED.
) Time: 795535 ns Iteration: 0 Process: /wrapper_tb/line__242 File: C:/Users/luks/Desktop/Schoolwork/EEL5721/git/Reconfig/Projec
```

As can be seen in the above images, both the simulation and software tests on the FPGA server functioned as intended. The software for the FPGA server was run multiple times to ensure that metastability issues were resolved. Additionally, one minor change was made to the testing software to ensure maximal coverage. The randomization used in C++ by default will choose a seed upon compilation, meaning that the same pseudo-random tests would be used every time the test software was run unless the code was recompiled. To optimize the testing process and avoid having to recompile the tests every time and ensure truly random number sequences, the main.cpp file within the “sw” folder of “dram_test” was modified to include “srand(time(NULL));” at line 100 before the for loop is initialized. This line ensures the random seed used is the number of seconds controlled by the server’s clock. Since this is a variable, it

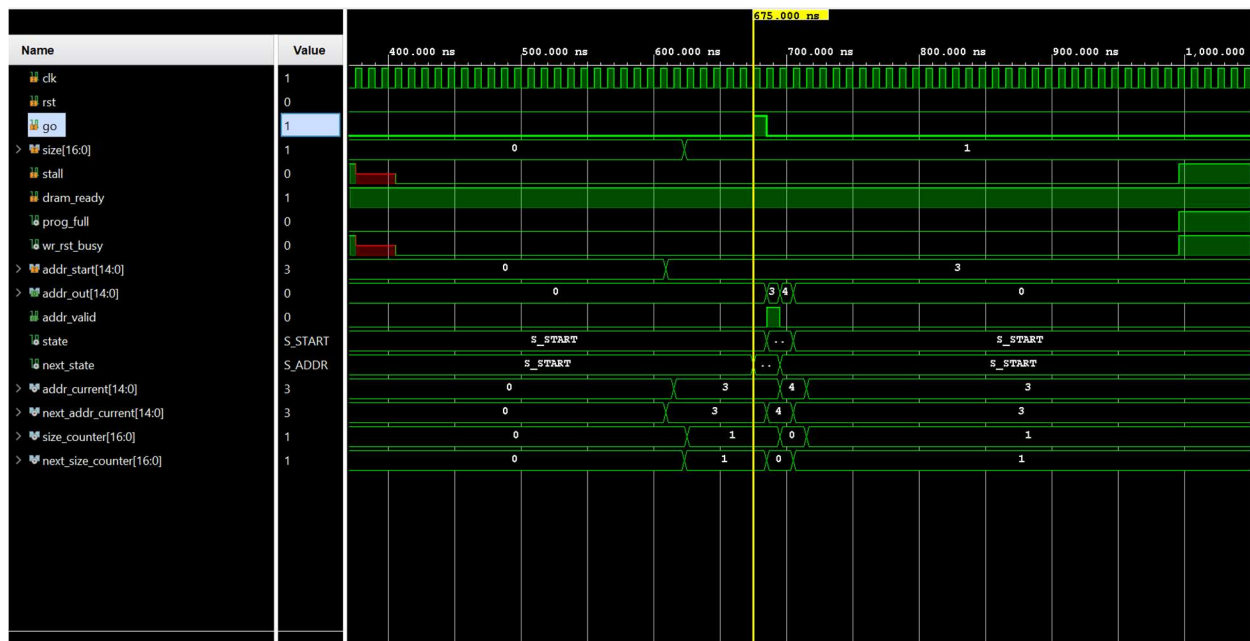
changes upon every run and therefore means every run of the program will produce different random tests without having to recompile the software.

Address Generator

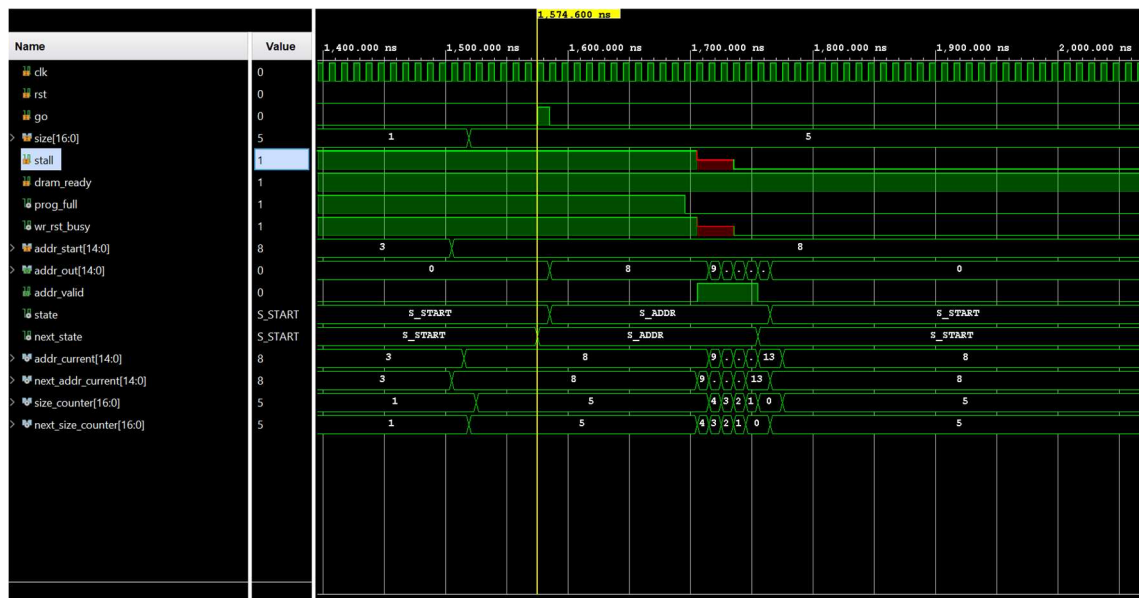
The address generator, upon seeing its “go” signal (the “rcv” signal of the handshake) set to high, will begin to generate addresses. The address generator is built as a 2-process finite state machine based on the provided address generator code from the website. In the clocked process, the current address, size counter, and state signals are registered. In the combinational process, the first state is a start state. This state waits for go to be asserted and assigns the next address and size to be the starting address and input size, respectively. Given go is set to high, it will set the next state to the address computation state. In the address computation state, the output address is set to be the current address. Then, a conditional checks for one of three cases:

1. If the size counter is set to 0, then the state machine restarts
2. If the stall is set high, hold the current address
3. Otherwise, a normal operation ensues. The valid signal is set high, the next address is set to the current address +1, and the next size counter is set to the current size counter -1.

The results of this program are shown in the testbench image below.



This screenshot shows how a go signal starts address generations, given the stall signal is low. For a size of 1 and a start address of 3, the address generator produces addresses 3 and 4, then restarts. The stall signal does temporarily have an unknown value due to the wr_rst_busy signal, but this only occurs when switching to 0 and the address generator does produce the correct output since it only stalls when the stall signal is high specifically. Because this was an issue with the FIFO itself, I was not able to fix it and it caused no issues with the simulator or the board itself.



The screenshot above shows how the addresses will not increment when the stall signal is high. Even though go had been asserted and the state machine transitioned to the next state, the stall signal being set to high prevents any address generation. After the stall is set to low, address generation occurs as expected.

FIFO

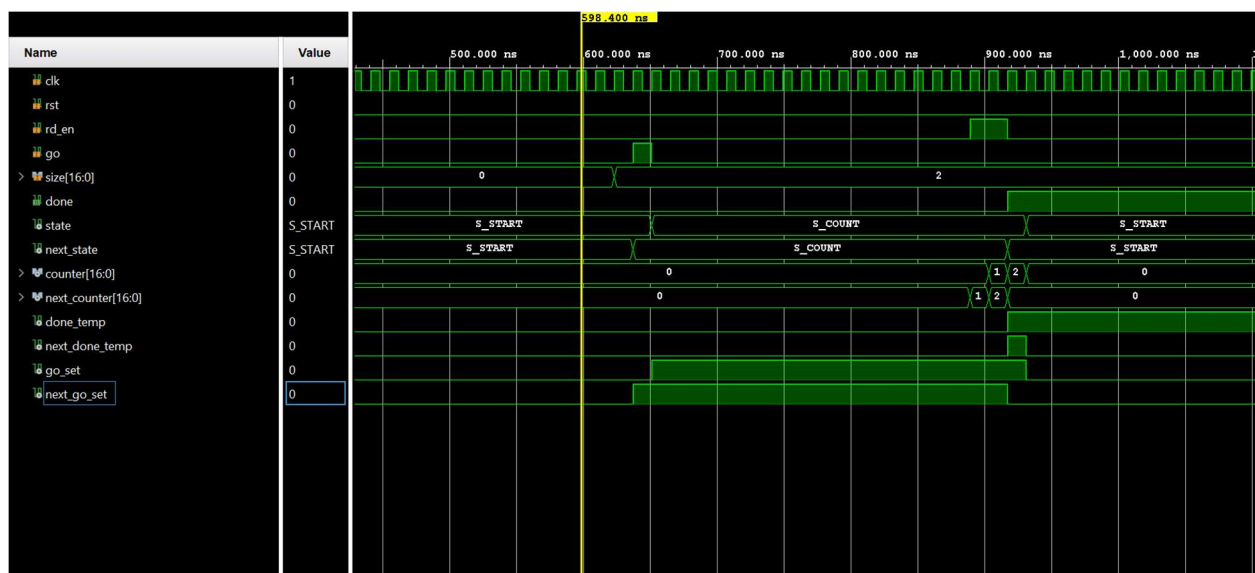
The FIFO was implemented using the FIFO generator by following the instructions from Dr. Stitt. Several of its signals drive logic within the dram_rd entity, which can be seen in the block diagram at the beginning of this section. This entity was used at the instruction of Dr. Sitt's lecture videos.

Handshake/Dual Flop

The metastability correcting components of this entity, the handshake and the dual flop, were both implemented using the entities from lab 5 since they have already been tested. These entities were chosen both based on the advice of Dr. Stitt and the specifications of the data crossing. For instance, the handshake was chosen to send multiple bits across the clock domains without caring much about throughput, while the dual flop was chosen to send single bits across the clock domains.

Counter

The counter is also implemented as a two-process finite state machine. The clocked process drives the register logic, and the combinational process drives the logic updating the registers. The main registers are the done, go_set, counter, and state registers. When go is set high, the go_set register is updated to high as well to ensure that the process continues to count throughout the entire time it should. The process counts if go_set is high, then stops counting when the counter register reaches the size input. The following image shows a successful test run.



As can be seen in the image above, when go is asserted the go_set register is updated and the state is updated to the count state. Only upon a read enable does the count begin to increment, and done is asserted after two counts. After this process, there is a successful restart with the register values being reset.